

Generative models

November 4, 2025

Generative models are a class of machine learning models designed to generate new data samples that resemble a given set of training data.

These models learn the underlying patterns and structures in the training data and can generate new data points that are statistically similar to the training data. Generative models have a wide range of applications in various fields, including image generation, text generation etc.

Given training data, generate new samples from same distribution



Training data $\sim p_{\text{data}}(x)$

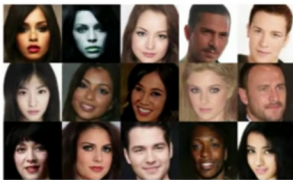


Generated samples $\sim p_{\text{model}}(x)$

Want to learn $p_{\text{model}}(x)$ similar to $p_{\text{data}}(x)$

Why Generative Models?

- Realistic samples for artwork, super-resolution, colorization, etc.



Generative Adversarial Networks

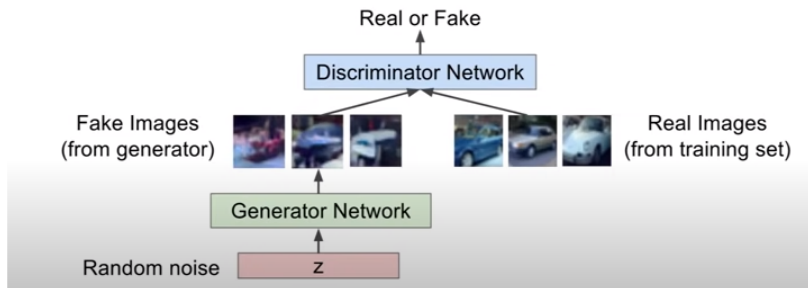
Is a generative model in which we sample from a simple distribution, e.g. random noise, and learn a transformation to training distribution.

And we use neural networks to learn this complex transformation.

Training GANs: Two-player game

Generator network: try to fool the discriminator by generating real-looking images

Discriminator network: try to distinguish between real and fake images



Here we usually take $z \sim N(0, I)$.

$N(0, I)$, represents a multivariate Gaussian distribution where each variable has a mean of zero (0) and a variance of one (1), and the variables are uncorrelated.

Train jointly in **minimax game**

Minimax objective function: Discriminator outputs likelihood in (0,1) of real image

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log \underbrace{D_{\theta_d}(x)}_{\substack{\text{Discriminator output} \\ \text{for real data } x}} + \mathbb{E}_{z \sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\substack{\text{Discriminator output for} \\ \text{generated fake data } G(z)}}) \right]$$

Discriminator (θ_d) wants to maximize objective such that $D(x)$ is close to 1 (real) and $D(G(z))$ is close to 0 (fake)

Generator (θ_g) wants to minimize objective such that $D(G(z))$ is close to 1 (discriminator is fooled into thinking generated $G(z)$ is real)

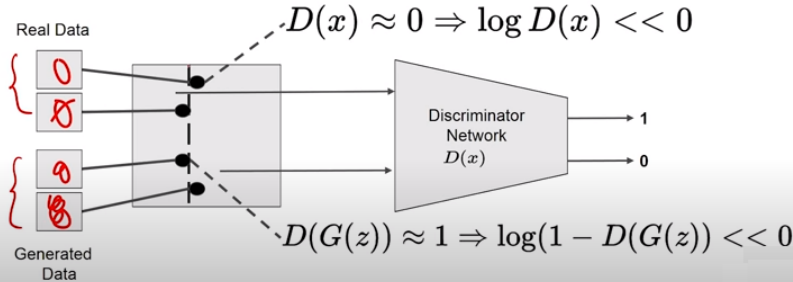
Training Discriminator Network

Before Training

$$\max_D [\mathbb{E}_{x \sim P_{data}(x)} \log(D(x)) + \mathbb{E}_{z \sim P_z(z)} \log(1 - D(G(z)))]$$

$D(x)$ should be 1

$D(G(z))$ should be 0



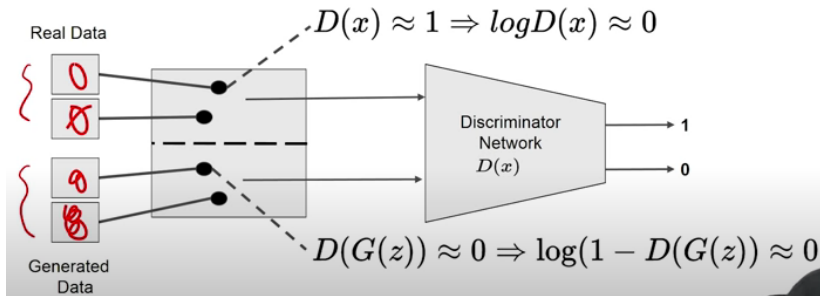
Training Discriminator Network

After Training

$$\max_D [\mathbb{E}_{x \sim P_{data}(x)} \log(D(x)) + \mathbb{E}_{z \sim P_z(z)} \log(1 - D(G(z)))]$$

$D(x)$ should be 1

$D(G(z))$ should be 0



Training Generator Network

$$\min_G [\mathbb{E}_{x \sim P_{data}} (\log D(x)) + \mathbb{E}_{z \sim P(z)} (\log(1 - D(G(z))))]$$

$$\Rightarrow \min_G [\mathbb{E}_{z \sim P(z)} (\log(1 - D(G(z))))]$$

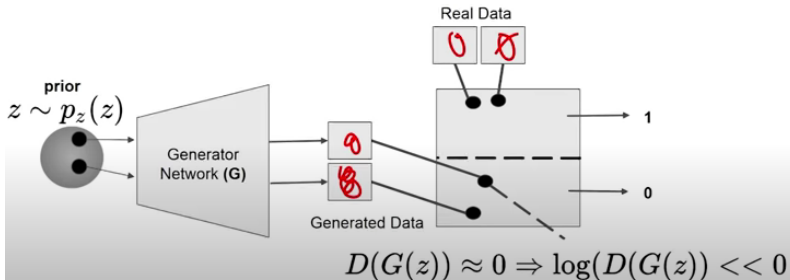
here we must have minimizing the chance of discriminator correctly classifying fake as fake.

Training Generator Network

Before Training

$$\max_G [\mathbb{E}_{z \sim P_z(z)} \log(D(G(z)))]$$

$D(G(z))$ should be 1

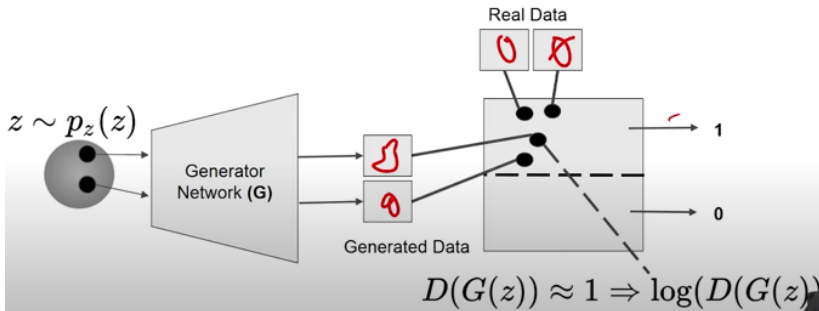


Training Generator Network

After Training

$$\max_G [\mathbb{E}_{z \sim P_z(z)} \log(D(G(z)))]$$

$D(G(z))$ should be 1



procedure GAN TRAINING

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$
- Sample minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from data generating distribution $p_{data}(\mathbf{x})$
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta} \frac{1}{m} \sum_{i=1}^m \left[\log D_{\theta} \left(x^{(i)} \right) + \log \left(1 - D_{\theta} \left(G_{\phi} \left(z^{(i)} \right) \right) \right) \right]$$

end for

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$
- Update the generator by ascending its stochastic gradient

$$\nabla_{\phi} \frac{1}{m} \sum_{i=1}^m \left[\log \left(D_{\theta} \left(G_{\phi} \left(z^{(i)} \right) \right) \right) \right]$$

end for

end procedure